AlertaBairro: alertas de chuva forte e calor extremo por bairro

Global Solution 2026 - CitizenTech | Coding for Security | FIAP

Aluno(a): Enrico Cavalieri Lee RM:569531

1. Problema e cidadão

Moradores de bairros em áreas de risco (beiras de rio, várzeas e encostas) costumam saber do temporal ou da onda de calor quando ele já chegou. O IBGE, com base no Censo 2010, estimou **8,3 milhões de pessoas** vivendo em áreas com risco de enchentes e deslizamentos no Brasil. O calor extremo também cresce: levantamento divulgado em 2023 com dados do INMET indica que os dias de onda de calor no país passaram de cerca de **7 para 52 por ano** em seis décadas. Os avisos oficiais existem, mas nem sempre chegam ao nível do bairro, especialmente a idosos, pessoas sem smartphone ou com pouca conectividade.

Cidadão beneficiado: moradores de bairros vulneráveis e os pontos de apoio da comunidade (associação de moradores, escola, UBS), que repassam o aviso a quem não está conectado. **Evidência:** dados públicos do IBGE e do INMET citados acima e os casos recorrentes de alagamento em bairros como o Jardim Pantanal (São Paulo) e Sarandi (Porto Alegre), usados como dados de exemplo no protótipo.

Solução: o AlertaBairro cadastra bairros monitorados, consulta a previsão do tempo de cada um e, quando a chuva ou a temperatura previstas passam dos limiares definidos, emite alertas em três níveis (ATENÇÃO, ALERTA, EMERGÊNCIA) com orientação prática e telefones de emergência (Defesa Civil 199, SAMU 192). Os alertas ficam na página web e são enviados em tempo real, por sockets TCP, a receptores instalados nos pontos de apoio do bairro.

2. ODS relacionados

ODS	Contribuição
11 - Cidades e comunidades sustentáveis	Meta 11.5: reduzir mortes e pessoas afetadas por desastres, com aviso antecipado por bairro.
13 - Ação contra a mudança do clima	Meta 13.1: fortalecer a resiliência a eventos climáticos extremos (chuvas intensas e calor).

ODS	Contribuição
3 - Saúde e bem-estar	Orientações contra exaustão por calor, voltadas sobretudo a idosos e crianças.

3. Arquitetura e modelo de dados

Aplicação em Python organizada em módulos com responsabilidade única e tratamento de exceções em todas as camadas:

Módulo	Função
`app.py`	API REST Flask: GET, POST, PUT e DELETE em /api/bairros, verificação de clima e histórico de alertas; tratadores de erro padronizados.
`db.py`	CRUD no MySQL sem ORM, com mysql-connector e SQL parametrizado; commit/rollback e fechamento da conexão em finally.
`validacao.py`	Validação de toda entrada: campos permitidos, regex para textos, faixas de coordenadas, UF e tipo de risco em listas fechadas.
`clima.py`	Fonte externa: Open-Meteo via requests e JSON, com timeout e conferência do formato da resposta.
`regras.py`	Converte a previsão em alertas por limiares de chuva (30/50/100 mm) e temperatura máxima (32/35/38 °C).
`socket_servidor.py` / `cliente_receptor.py`	Módulo de redes: servidor TCP multithread com protocolo INSCREVER <id> e cliente que exibe os alertas.
`logger.py`	Log em arquivo com data/hora, ação e resultado.
`templates/` e `static/`	Interface web mínima (HTML/CSS/JS) que consome a própria API.

Fluxo: navegador -> API Flask -> MySQL; ao verificar um bairro, a API consulta a Open-Meteo, aplica as regras, grava os alertas e os publica no servidor TCP, que os entrega aos receptores inscritos naquele bairro.

Modelo de dados (MySQL): `bairros` (id, nome, cidade, uf, latitude, longitude, area_de_risco, criado_em, atualizado_em), com UNIQUE(nome, cidade, uf); e `alertas` (id, bairro_id FK, tipo, nivel, valor, data_previsao, mensagem, criado_em), com ON DELETE CASCADE e UNIQUE(bairro_id, tipo, data_previsao), para que uma nova verificação atualize o alerta do dia em vez de duplicá-lo. Relação 1:N. **Nenhuma tabela guarda dados pessoais**: o sistema monitora lugares, não pessoas.

4. Segurança e privacidade

Controles aplicados: credenciais apenas no `.env` (com `.env.example` sem segredos e .env no .gitignore); usuário MySQL de privilégio mínimo (só SELECT, INSERT, UPDATE e DELETE); consultas 100% parametrizadas; validação de toda entrada, inclusive limite de 16 KB no corpo e comandos do socket validados por regex e limitados a 64 bytes; mensagens de erro genéricas, com o detalhe técnico só no log; log sem senhas nem dados pessoais e sem quebras de linha (evita forjar registros); Flask com debug desligado; cabeçalhos de segurança (CSP, X-Frame-Options) e limite de verificações por bairro (HTTP 429).

LGPD: princípio da necessidade (art. 6º, III). O sistema não coleta nome, telefone, CPF ou localização de pessoas; campos não previstos enviados à API são recusados; a demonstração usa apenas dados fictícios ou abertos (coordenadas públicas aproximadas).

Ameaça	Impacto	Mitigação
1. SQL Injection - atacante envia, por exemplo, `x' OR 1=1--` no nome do bairro.	Leitura, alteração ou exclusão de dados do banco.	Todas as consultas usam parâmetros (%s); a entrada é validada por regex e lista branca antes de chegar ao banco; o usuário do MySQL não pode DROP nem ALTER.
2. Vazamento de informação em erros - falha de banco ou exceção expõe SQL, caminhos ou versões.	Facilita o reconhecimento do sistema para novos ataques.	Tratadores de erro devolvem só mensagens genéricas (400, 404, 503, 500); o detalhe vai para o log, que fica no servidor.

Ameaça	Impacto	Mitigação
3. Dados externos ou de rede maliciosos - resposta inesperada da API de clima ou cliente TCP enviando comandos inválidos ou enormes.	Travamento, alertas falsos ou consumo excessivo de memória.	Timeout de 10 s e verificação de tipos e tamanhos da resposta da Open-Meteo (falha vira 502); intervalo mínimo de 30 s por bairro (429); no socket, regex, limite de 64 bytes, timeout de inscrição e limite de 100 clientes.

5. Validação e viabilidade

O protótipo foi executado com MySQL real e testado endpoint a endpoint (navegador e curl), seguindo o roteiro do README, além de **17 testes automatizados** com mock (todos aprovados). Resultados: cadastro válido -> 201; duplicado -> 409; tentativa de injeção ou dados fora de faixa -> 400 sem gravar nada; edição -> 200; exclusão -> 204 e, repetida, -> 404; verificação com previsão de 112 mm de chuva -> alerta de EMERGÊNCIA gravado e entregue ao receptor TCP; segunda verificação seguida -> 429 com Retry-After; banco desligado -> 503 sem detalhes técnicos; API de clima indisponível -> 502. Cada ação ficou registrada no log com data, hora e resultado. Durante os testes foi encontrado e corrigido um erro de acentuação na carga inicial do banco ("SÃ£o Paulo"), resolvido com `SET NAMES utf8mb4` no script SQL.

Viabilidade: usa apenas software livre e uma API gratuita sem chave; roda em um computador simples. O ecossistema natural envolve a Defesa Civil municipal (que calibraria os limiares por bairro), associações de moradores, escolas e UBS como pontos de recepção, e operadoras de telecomunicação para ampliar a entrega.

6. Limitações

A verificação é disparada manualmente (não há agendamento automático); não há autenticação, por isso o servidor escuta apenas em 127.0.0.1; o socket não usa criptografia (TLS); o limite de verificações fica em memória e zera ao reiniciar; e os limiares são genéricos, não calibrados localmente. Próximos passos: agendamento periódico, login para operadores da Defesa Civil, TLS e envio por SMS para quem não tem internet.

7. Recursos extras (bônus)

Recursos além do conteúdo da disciplina, adotados para reforçar a segurança e a confiabilidade:

Recurso	Por que foi usado
Cabeçalhos HTTP de segurança (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options)	Segunda camada de defesa: mesmo que algum dado malicioso escape da validação, o navegador só executa scripts do próprio site (contra XSS) e a página não pode ser embutida em outro site (contra clickjacking).
Limite de requisições (rate limit) com HTTP 429 e Retry-After	Impede que alguém use o sistema para disparar consultas em massa à API de clima (abuso/DoS) e evita alertas repetidos em sequência.
Testes automatizados com unittest.mock	Comprovam validação, códigos HTTP, regras de alerta e socket de forma repetível, sem depender de banco ou internet; são evidência da etapa de validação.
RotatingFileHandler no log	Limita o log a 5 arquivos de 1 MB, evitando que o disco encha (o próprio log poderia virar alvo de DoS).
@contextmanager no acesso ao banco	Centraliza commit, rollback e fechamento da conexão, evitando conexões abertas ou transações pela metade quando ocorre erro.

8. Uso de IA generativa

Utilizei IA generativa (Claude, da Anthropic) como apoio nos pontos mais complexos do projeto: o servidor de sockets TCP multithread e seu protocolo de inscrição, a validação de entradas com expressões regulares, o tratamento e a validação da resposta da API externa e a depuração do erro de codificação UTF-8 no banco. Usei para a criação do READ.ME e para o site HTML, css e js. O relatório e foi criado pelo Claude e os commits do git foram feitos pelo Claude.

Referências

- IBGE. População em áreas de risco no Brasil (2018, base Censo 2010).
- Correio Braziliense. Número de dias com ondas de calor aumentou de 7 para 52 em 60 anos no Brasil (nov. 2023, dados INMET).
- Open-Meteo.com, Weather API, licença CC BY 4.0.
- ONU. Objetivos de Desenvolvimento Sustentável 3, 11 e 13.
- Lei nº 13.709/2018 (LGPD).